Lista de Exercícios — Método de Gauss–Seidel

Sistemas lineares em cálculo numérico

Felipe Zanata

Código (Google Colab): https://colab.research.google.com/github/Z4nata/Z4nata/blob/claude/new-session-62iy5i/calculo-numerico/gauss-seidel/gauss_seidel.ipynb

$$\begin{cases} 10x - y + 2z = 6 \\ -x + 11y - z = 22 \\ 2x - y + 10z = -10 \end{cases}$$

1. Forma matricial $A\mathbf{x} = \mathbf{b}$

$$\underbrace{\begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x \\ y \\ z \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 6 \\ 22 \\ -10 \end{bmatrix}}_{\mathbf{b}}$$

A é a matriz dos coeficientes, $\mathbf{x} = (x, y, z)^T$ é o vetor das incógnitas e $\mathbf{b} = (6, 22, -10)^T$ é o vetor dos termos independentes.

2. Dominância diagonal por linhas

A é estritamente diagonalmente dominante por linhas se, em toda linha i, $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$:

$$\begin{aligned} &\text{Linha 1:} \quad |10| = 10 \;>\; |-1| + |2| = 3 \\ &\text{Linha 2:} \quad |11| = 11 \;>\; |-1| + |-1| = 2 \\ &\text{Linha 3:} \quad |10| = 10 \;>\; |2| + |-1| = 3 \end{aligned}$$

As três desigualdades valem, então A **é estritamente diagonalmente dominante por linhas**.

Por que importa: essa propriedade é uma condição *suficiente* para que o método de Gauss–Seidel (e também o de Jacobi) convirja para a solução, qualquer que seja a aproximação inicial $\mathbf{x}^{(0)}$. Intuitivamente, ao isolar a incógnita da diagonal, cada nova componente depende pouco das outras (os coeficientes fora da diagonal, divididos por a_{ii}, são pequenos), então o erro é multiplicado por um fator menor que 1 a cada iteração. Aqui $\max_i \frac{1}{|a_{ii}|} \sum_{j \neq i} |a_{ij}| = \max\{0.3,\ 0.18,\ 0.3\} = 0.3 < 1$, uma folga grande, o que indica convergência rápida.

3. Fórmulas iterativas de Gauss–Seidel

Isolando x na 1ª equação, y na 2ª e z na 3ª:

$$\begin{aligned} x^{(k)} &= \frac{6 + y^{(k-1)} - 2z^{(k-1)}}{10} \\ y^{(k)} &= \frac{22 + x^{(k)} + z^{(k-1)}}{11} \\ z^{(k)} &= \frac{-10 - 2x^{(k)} + y^{(k)}}{10} \end{aligned}$$

Valores usados em cada iteração:

• $x^{(k)}$ usa y e z da iteração anterior, $y^{(k-1)}$ e $z^{(k-1)}$ (ainda não há valores novos).

• $y^{(k)}$ já usa o $x^{(k)}$ *recém-calculado* e o $z^{(k-1)}$ antigo.

• $z^{(k)}$ usa $x^{(k)}$ e $y^{(k)}$, ambos da iteração atual.

Essa é a diferença para o método de Jacobi, que só usa valores da iteração $k-1$: Gauss–Seidel aproveita cada valor assim que ele é calculado.

4. Três primeiras iterações à mão, com $\mathbf{x}^{(0)} = (0, 0, 0)$

Valores exibidos com 6 casas decimais; nas contas foram mantidas todas as casas dos valores anteriores.

Iteração $k = 1$

Usa $\mathbf{x}^{(0)} = (0.000000,\ 0.000000,\ 0.000000)$.

$$x^{(1)} = \frac{6 + y^{(0)} - 2z^{(0)}}{10} = \frac{6 + 0.000000 - 2 \cdot 0.000000}{10} = \frac{6.000000}{10} = \boxed{0.600000}$$

$$y^{(1)} = \frac{22 + x^{(1)} + z^{(0)}}{11} = \frac{22 + 0.600000 + 0.000000}{11} = \frac{22.600000}{11} = \boxed{2.054545}$$

$$z^{(1)} = \frac{-10 - 2x^{(1)} + y^{(1)}}{10} = \frac{-10 - 2 \cdot 0.600000 + 2.054545}{10} = \frac{-9.145455}{10} = \boxed{-0.914545}$$

$$\|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|_\infty = \max\{0.600000,\ 2.054545,\ 0.914545\} = 2.054545$$

Iteração $k = 2$

Usa $\mathbf{x}^{(1)} = (0.600000,\ 2.054545,\ -0.914545)$.

$$x^{(2)} = \frac{6 + y^{(1)} - 2z^{(1)}}{10} = \frac{6 + 2.054545 - 2 \cdot (-0.914545)}{10} = \frac{9.883636}{10} = \boxed{0.988364}$$

$$y^{(2)} = \frac{22 + x^{(2)} + z^{(1)}}{11} = \frac{22 + 0.988364 + (-0.914545)}{11} = \frac{22.073818}{11} = \boxed{2.006711}$$

$$z^{(2)} = \frac{-10 - 2x^{(2)} + y^{(2)}}{10} = \frac{-10 - 2 \cdot 0.988364 + 2.006711}{10} = \frac{-9.970017}{10} = \boxed{-0.997002}$$

$$\|\mathbf{x}^{(2)} - \mathbf{x}^{(1)}\|_\infty = \max\{0.388364,\ 0.047835,\ 0.082456\} = 0.388364$$

Iteração $k = 3$

Usa $\mathbf{x}^{(2)} = (0.988364,\ 2.006711,\ -0.997002)$.

$$x^{(3)} = \frac{6 + y^{(2)} - 2z^{(2)}}{10} = \frac{6 + 2.006711 - 2 \cdot (-0.997002)}{10} = \frac{10.000714}{10} = \boxed{1.000071}$$

$$y^{(3)} = \frac{22 + x^{(3)} + z^{(2)}}{11} = \frac{22 + 1.000071 + (-0.997002)}{11} = \frac{22.003070}{11} = \boxed{2.000279}$$

$$z^{(3)} = \frac{-10 - 2x^{(3)} + y^{(3)}}{10} = \frac{-10 - 2 \cdot 1.000071 + 2.000279}{10} = \frac{-9.999864}{10} = \boxed{-0.999986}$$

$$\|\mathbf{x}^{(3)} - \mathbf{x}^{(2)}\|_\infty = \max\{0.011708,\ 0.006432,\ 0.002985\} = 0.011708$$

Após três iterações, $\mathbf{x}^{(3)} \approx (1.000071,\ 2.000279,\ -0.999986)$, já muito próximo da solução exata $(1, 2, -1)$. O erro entre iterações caiu de 2.054545 para 0.011708.

5. Implementação em Python

Código completo no Colab: abrir notebook. Como o vetor `x` é atualizado no lugar, ao calcular a componente `i` as posições `x[:i]` já têm os valores da iteração k e `x[i+1:]` ainda têm os da iteração $k-1$, exatamente como nas fórmulas da questão 3.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

A = np.array([[10, -1,  2],
              [-1, 11, -1],
              [ 2, -1, 10]], dtype=float)
b = np.array([6, 22, -10], dtype=float)
x0 = np.zeros(3)

def gauss_seidel(A, b, x0, tol=1e-5, max_iter=100):
    """Retorna as aproximações x^(0), ..., x^(k) e os erros ||x^(k) - x^(k-1)||_inf."""
    x = np.array(x0, dtype=float)
    aproximacoes = [x.copy()]
    erros = [np.nan]  # não há erro na iteração 0

    for k in range(1, max_iter + 1):
        x_anterior = x.copy()
        for i in range(len(b)):
            soma = A[i, :i] @ x[:i] + A[i, i + 1:] @ x[i + 1:]
            x[i] = (b[i] - soma) / A[i, i]

        erro = np.max(np.abs(x - x_anterior))
        aproximacoes.append(x.copy())
        erros.append(erro)
        if erro < tol:
            return np.array(aproximacoes), np.array(erros)

    raise RuntimeError(f"Não convergiu em {max_iter} iterações")

aproximacoes, erros = gauss_seidel(A, b, x0, tol=1e-5)

n_iter = len(aproximacoes) - 1
x, y, z = aproximacoes[-1]

print(f"Iterações realizadas: {n_iter}")
print(f"Solução aproximada: x = {x:.8f}, y = {y:.8f}, z = {z:.8f}")
print(f"Erro da última iteração: {erros[-1]:.3e}")
print(f"Conferência (np.linalg.solve): {np.linalg.solve(A, b)}")
```

SAÍDA

```
Iterações realizadas: 6
Solução aproximada: x = 1.00000003, y = 1.99999998, z = -1.00000001
Erro da última iteração: 1.263e-06
Conferência (np.linalg.solve): [ 1.  2. -1.]
```

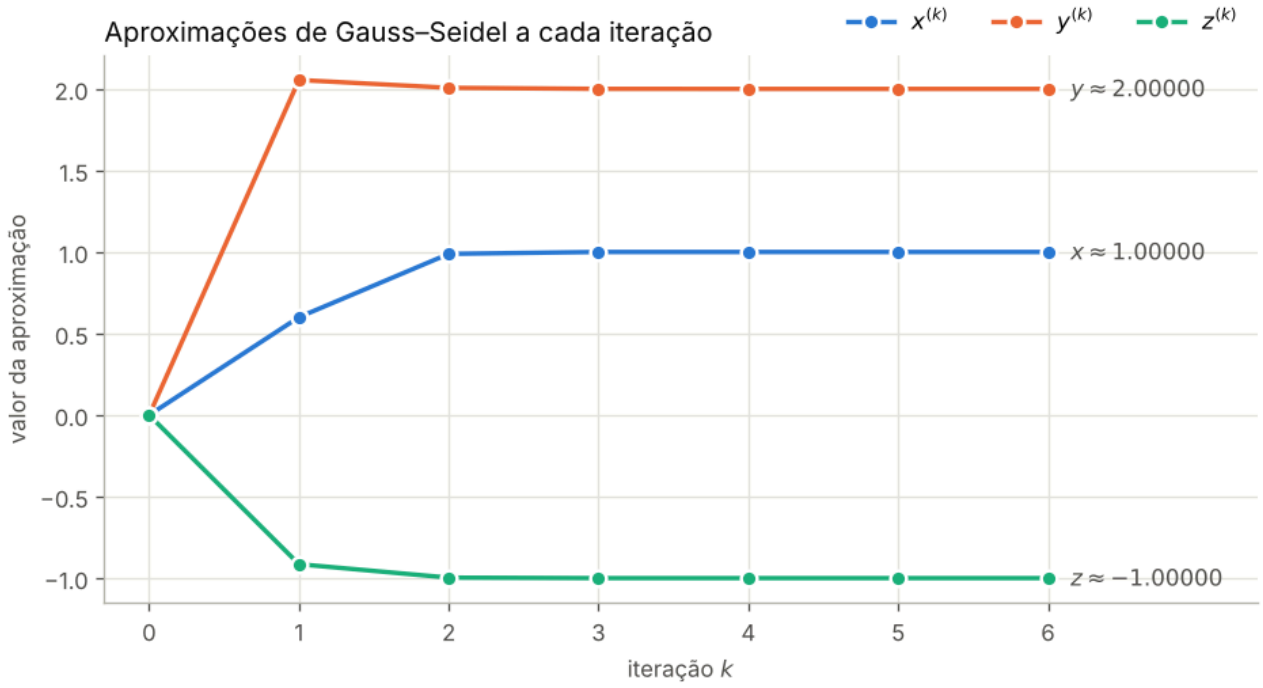
> O critério $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_\infty < 10^{-5}$ foi atingido em **6 iterações**, com solução aproximada $\mathbf{x}^{(6)} \approx (1.00000003,\ 1.99999998,\ -1.00000001)$. A solução exata do sistema é $(1, 2, -1)$.

6. Tabela das iterações

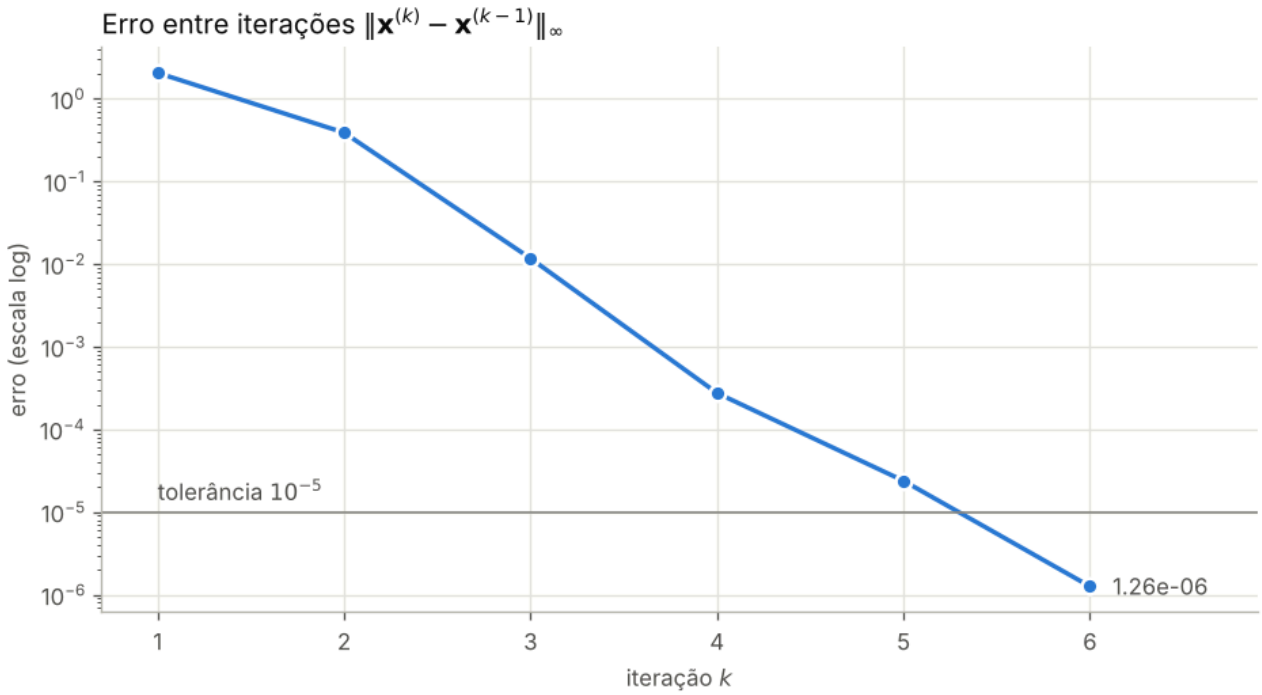
k	$x^{(k)}$	$y^{(k)}$	$z^{(k)}$	$\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_\infty$
0	0.00000000	0.00000000	0.00000000	—
1	0.60000000	2.05454545	−0.91454545	2.055×10^{0}
2	0.98836364	2.00671074	−0.99700165	3.884×10^{-1}
3	1.00007140	2.00027907	−0.99998637	1.171×10^{-2}
4	1.00002518	2.00000353	−1.00000468	2.755×10^{-4}
5	1.00000129	1.99999969	−1.00000029	2.389×10^{-5}
6	1.00000003	1.99999998	−1.00000001	1.263×10^{-6}

As linhas $k = 1, 2, 3$ coincidem com as contas feitas à mão na questão 4.

7. Gráficos



(a) $x^{(k)}$, $y^{(k)}$ e $z^{(k)}$ em função da iteração k.



(b) Erro entre iterações $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_\infty$ em escala logarítmica.

Análise. As três componentes se aproximam de $(1, 2, -1)$ já na 3ª iteração e, daí em diante, as curvas ficam praticamente sobre os valores exatos. Em escala logarítmica o erro desce quase em linha reta: cada iteração reduz o erro entre uma e duas ordens de grandeza (convergência linear rápida), coerente com a forte dominância diagonal de A vista na questão 2. A tolerância 10^{-5} é ultrapassada entre $k = 5$ e $k = 6$.